

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- d) Display the PIDs of parent and child processes.

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano task2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    // Read command from user
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    // Create child process
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    // Child process
    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        // Execute the command
        execlp(command, command, NULL);

        // Executes only if exec fails
        perror("Execution failed");
        exit(1);
    }

    // Parent process
    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        // Wait for child process
        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

1)Then press ctrl+o enter ctrl+x then gcc filename.c -o filename

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano task2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc task2.c -o task2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano task2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc task2.c -o task2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./task2
Enter a Linux command: |
```

3)then ./filename

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano task2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc task2.c -o task2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./task2
Enter a Linux command: ls

Parent Process
Parent PID : 679
Child PID  : 698

Child Process
Child PID  : 698
Parent PID : 679
linux myprogram problem1 problem1.c problem1.txt sum1 sum1.c task2 task2.c teja vishwa vishwa.c

Child process completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

4)enter the linux command

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware

Linux Commands Used

1. uname

The `uname` command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ uname
Linux
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

2. lscpu

The `lscpu` command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./task2
Enter a Linux command: lscpu

Parent Process
Parent PID : 747
Child PID  : 750

Child Process
Child PID  : 750
Parent PID : 747

Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               39 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      16
On-line CPU(s) list:        0-15
Vendor ID:                   GenuineIntel
Model name:                  13th Gen Intel(R) Core(TM) i7-13620H
CPU family:                  6
Model:                      186
Thread(s) per core:         2
Core(s) per socket:         8
Socket(s):                   1
Stepping:                    2
BogoMIPS:                    5836.79
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_relia
ble nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3d
nowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsav
ec xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabilities

Virtualization features:
Virtualization:              VT-x
Hypervisor vendor:           Microsoft
Virtualization type:         full

Caches (sum of all):
L1d:                          384 KiB (8 instances)
L1i:                          256 KiB (8 instances)
L2:                            10 MiB (8 instances)
L3:                           24 MiB (1 instance)

NUMA:
NUMA node(s):                1
NUMA node0 CPU(s):          0-15

Vulnerabilities:
Gather data sampling:        Not affected
Ghostwrite:                  Not affected
Indirect target selection:    Not affected
Itlb multihit:               Not affected
L1tf:                        Not affected
Mds:                         Not affected
Meltdown:                    Not affected
Mmio stale data:              Not affected
Old microcode:                Not affected
Reg file data sampling:       Mitigation; Clear Register File
Retbleed:                     Mitigation; Enhanced IBRS
Spec rstack overflow:         Not affected
Spec store bypass:            Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:                   Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:                   Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSB-eIBRS SW sequence; BHI BHI_DIS_S
Srbds:                        Not affected
Tsa:                          Not affected
Tsx async abort:              Not affected
Vmscape:                      Not affected

Child process completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

3. ps

The `ps` command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ps
PID TTY          TIME CMD
 319 pts/0        00:00:00 bash
 779 pts/0        00:00:00 ps
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ top
top - 03:13:42 up 13 min,  1 user,  load average: 0.00, 0.01, 0.00
Tasks: 24 total,  1 running, 23 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  7755.6 total,  7143.1 free,   583.7 used,   180.7 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used.  7172.0 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24136	15412	11620	S	0.0	0.2	0:00.86	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.0	0:00.01	init-systemd(Ub
7	root	20	0	3180	2048	1940	S	0.0	0.0	0:00.00	init
47	root	19	-1	50392	16660	15360	S	0.0	0.2	0:00.18	systemd-journal
81	systemd+	20	0	22540	14640	12020	S	0.0	0.2	0:00.06	systemd-resolve
91	root	20	0	35408	12368	9228	S	0.0	0.2	0:00.59	systemd-udev
113	root	20	0	2888	1948	1820	S	0.0	0.0	0:00.03	chronyd-starter
120	root	20	0	4472	3164	2916	S	0.0	0.0	0:00.01	cron
128	message+	20	0	8816	5344	4664	S	0.0	0.1	0:00.05	dbus-daemon
165	root	20	0	44092	29704	14564	S	0.0	0.4	0:00.23	networkd-dispat
175	root	20	0	18436	9416	8168	S	0.0	0.1	0:00.07	systemd-logind
192	syslog	20	0	220548	5668	4364	S	0.0	0.1	0:00.07	rsyslogd
221	_chrony	20	0	24456	11860	7088	S	0.0	0.1	0:00.09	chronyd
235	root	20	0	123456	32520	17904	S	0.0	0.4	0:00.19	unattended-upgr
236	_chrony	20	0	12092	2460	1816	S	0.0	0.0	0:00.00	chronyd
263	root	20	0	5492	2736	2548	S	0.0	0.0	0:00.01	agetty
316	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00	SessionLeader
317	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.03	Relay(319)
319	vishwa_+	20	0	6268	5532	3864	S	0.0	0.1	0:00.10	bash
321	root	20	0	8644	5564	4724	S	0.0	0.1	0:00.01	login
386	vishwa_+	20	0	22340	13564	10944	S	0.0	0.2	0:00.13	systemd
388	vishwa_+	20	0	22912	4084	2084	S	0.0	0.1	0:00.00	(sd-pam)
416	vishwa_+	20	0	6136	5480	3928	S	0.0	0.1	0:00.04	bash
793	vishwa_+	20	0	8208	6044	3900	R	0.0	0.1	0:00.00	top

